CRUCIBLE PLATFORM SERIES · JUNE 2026

Virtual Prototyping for RISC-V Silicon: From Idea to Validated Software on a Virtual SoC

Boot Linux on a RISC-V virtual prototype, integrate a custom AI accelerator IP, and run a quantized MNIST classifier. Zero physical hardware.

EXECUTIVE SUMMARY

Executive Summary

The semiconductor industry faces a fundamental timing problem: software development cannot begin until hardware exists, yet software readiness is the single largest determinant of time-to-market after tape-out. Virtual Prototyping (VP) breaks this dependency by enabling full-system simulation of SoCs — from bare-metal firmware through RTOS to full Linux — long before physical hardware exists.

This whitepaper presents a complete, working demonstration of VP applied to custom AI accelerator development. Using entirely open-source tools, we boot Linux on a RISC-V virtual prototype, integrate a custom accelerator IP modeled in software, and run a quantized MNIST neural network classifier — achieving 100% accuracy across 740 accelerator invocations performing over one million simulated compute cycles. All of this runs on a virtual SoC with a custom accelerator that exists only as a behavioral model. No silicon. No FPGA. No physical hardware of any kind.

The demonstration validates a core thesis: **with the right platform infrastructure, hardware-software co-development workflows become accessible, reproducible, and manageable from day one of the design cycle.**

TABLE OF CONTENTS

01	The Problem: Sequential Hardware-Software Development	3
02	Virtual Prototyping as the Solution	4
03	Reference Architecture: LiteX + VexRiscv SoC	5
04	Demo 1: Running Custom Software on a Virtual RISC-V Linux System	6
05	Demo 2: Custom AI Accelerator IP Integration	7
06	Demo 3: MNIST Neural Network Inference on the Virtual Accelerator	9
07	Crucible: Making VP Workflows Reproducible	10
08	Running the Demos	11
09	Conclusion	13
10	Glossary	14

© 2026 Igny CLI · Ignytion IO

The Problem: Sequential Hardware-Software Development

Traditional chip development follows a sequential timeline: architecture is defined, RTL is written and verified, the design goes through synthesis and physical implementation, silicon is fabricated, and only then can software teams begin real integration work on actual hardware. This sequential dependency means software development — often the longest pole in bringing a product to market — cannot start in earnest until the very end of the hardware cycle.

THE CONSEQUENCES

- **Late-stage integration bugs** — Issues like interrupt storms, bus contention, memory map conflicts, and boot sequence failures surface only during hardware-software integration, when fixes are expensive and time-consuming.
- **Idle software teams** — Firmware, driver, and application developers wait for silicon or FPGA prototypes, creating costly schedule gaps.
- **Limited design exploration** — Architectural trade-offs (bus topologies, memory maps, IP integration strategies) are committed to in RTL before they can be validated with real software workloads.
- **Non-reproducible test environments** — Physical hardware setups are difficult to replicate across teams, making regression testing fragile and collaboration harder.

For companies designing custom accelerator IPs — particularly in the AI/ML space — this sequential dependency is especially painful. The accelerator's value is only realized through software (ML frameworks, drivers, runtime libraries), yet that software cannot be meaningfully developed until the accelerator hardware is available.

Virtual Prototyping as the Solution

Virtual Prototyping addresses the sequential dependency by creating functionally accurate simulation models of complete SoCs. These models execute real software — the same firmware, the same drivers, the same operating systems — in a simulated environment that faithfully represents the target hardware's memory map, peripheral interfaces, and interrupt topology.

VP's value extends across the entire chip development lifecycle:

DEVELOPMENT PHASE	VP APPLICATION
Architecture Exploration	Evaluate bus topologies, memory maps, and IP integration strategies in hours rather than weeks using SystemC or Renode models
Parallel Software Development	Firmware, drivers, BSPs, and application software developed against a virtual model while RTL is still being designed
Custom IP De-risking	Functional models of accelerator or DSP IPs plugged into full-system VPs, enabling driver and workload development before RTL completion
System-Level Bug Detection	Interrupt conflicts, bus contention, memory map overlaps, and boot sequence failures caught during design phase
Regression Testing	Deterministic, reproducible simulations added to CI pipelines, creating a growing safety net of system-level tests
Post-Silicon Validation Prep	Validation test suites pre-developed against VP models, defining expected behavior before silicon arrives

Critically, VP environments are **deterministic** — a simulation that fails can be reproduced exactly, debugged with full observability, and automated. Unlike physical hardware, there are no flaky connections, no board-specific quirks, and no equipment scheduling conflicts.Let's see how Virtual Prototyping works in practice — on to the demos.

Reference Architecture: LiteX + VexRiscv SoC

SYSTEM SPECIFICATION

COMPONENT	DETAIL
CPU	VexRiscv RV32IMA with MMU (Linux-capable, SpinalHDL)
Bus	Wishbone interconnect
Memory	32 KB ROM · 32 KB SRAM · 16 KB Emulator RAM · 256 MB RAM
Peripherals	LiteUART · Timer0 · CPU Timer · SPI · Ignytion Accel
Guest OS	Linux 5.0.14 (Buildroot), Busybox userspace
Simulator	Renode 1.16.1 (open-source, by Antmicro)

MEMORY MAP

0x00000000
ROM (32 KB)
Boot code

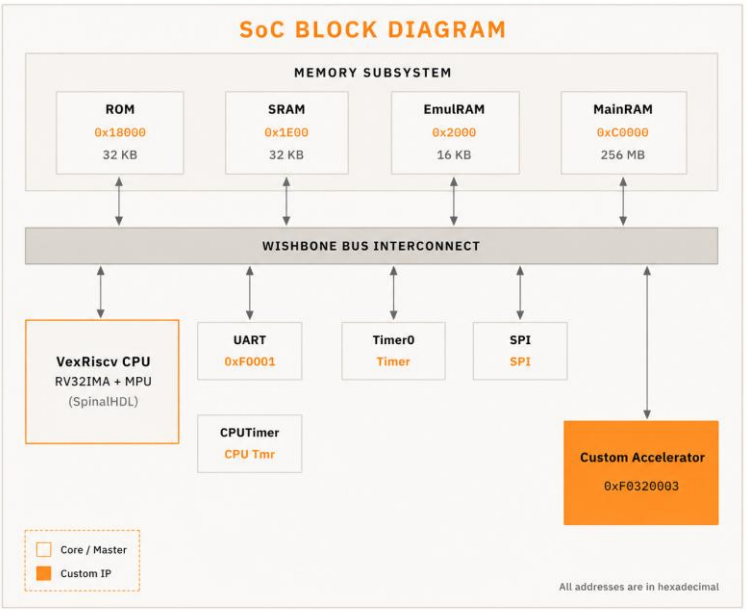
0x10000000
SRAM (32 KB)
Scratch memory

0x20000000
Emulator RAM (16 KB)
M-mode emulator

0xC0000000
Main RAM (256 MB)
Kernel, rootfs, DTB

0xF0001000
UART (LiteUART)
Serial console

0xF0010000
Ignytion Accel (Custom IP)
Accel · Expts 02–03



BOOT SEQUENCE

CPU @ 0x20000000 → **M-mode** Emulator → **Linux kernel** @0xC0000000 → **Unpack** rootfs → **Busybox** init → **Login** prompt

Demo 1: Running Custom Software on a Virtual RISC-V Linux System

AIM

Boot a complete Linux operating system on a simulated RISC-V SoC and execute custom, cross-compiled software on it — with zero physical hardware.

APPARATUS

ROLE	TOOL
Simulator	Renode 1.16.1
SoC Platform	LiteX + VexRiscv (RV32IMA, Linux-capable)
Guest OS	Linux 5.x (Buildroot)
Cross-Compiler	`riscv32-gcc via igny compiler install riscv32`
Interface	Igny CLI (Crucible Workbench)

PROCEDURE

```
01  cd virtual-prototyping-mnist-classifier
02  igny workspace sync
03  igny run renode --console scripts/litex_vexriscv_accel.resc -e s --hide-log
04  In the UART, log in as root (no password)
05  ./ignytion_banner
06  ./hello_world_ignytion
```

EXECUTION OUTPUT



RESULT

Linux boots to a login prompt on the virtual prototype. The ignytion_banner program reads platform information via the uname() syscall and reports “Platform: Linux-riscv32”, confirming execution on the simulated RISC-V CPU, not the host. The full write → cross-compile → deploy → run cycle works with no physical hardware. Experiment 01 passed.

Demo 2 — Adding a Custom AI Chip Part

We extended the virtual chip with a custom AI accelerator — a small chip peripheral that speeds up AI math — wrote software to drive it, and checked every answer. No physical chip. No finished hardware design. No specialized equipment.

This demo answers the question every silicon startup faces: *How do we start writing software for the chip before the physical hardware is built?*

WHAT THE ACCELERATOR DOES

- **Dot product** — multiply two lists of numbers together and add up the results. This is the core math inside nearly all AI.
- **Add** — add two lists of numbers together, item by item.
- **ReLU** — a simple rule: if a number is negative, make it zero. AI uses this constantly.

EXECUTION ON CRUCIBLE

```
root@buildroot:~# ./teroot@buildroot:~# ./test_accel

============================================================
   Ignytion Accelerator IP -- Functional Validation
============================================================
   Base: 0xF0010000  |  Signature: 0xDEADBEEF  |  Version: 0x49474E01

-- Test 1: Dot Product ---------------------------------------
   A = [1, 2, 3, 4]
   B = [5, 6, 7, 8]
   Expected: 70

   Result:   70 (correct)
   Cycles:   8

   PASS Dot product verified
```

RESULT

The custom chip peripheral was designed, added to the virtual chip, and controlled by ordinary software — before any physical hardware was built. Every operation the accelerator ran gave the right answer, and the same software would run on the real chip later with barely a change. Full pass/fail details are on the next slide.

WHY THIS MATTERS

- **For hardware teams** — your accelerator doesn't need to be finished hardware to start software work. Model its behavior, plug it into the virtual chip, and your software team can begin immediately.
- **For software teams** — you don't wait for silicon. The virtual chip talks to software the same way the real chip will, so this test program would run on real hardware with barely a change.

SUPPORTED OPERATIONS

MODE	CODE	INPUT	OUTPUT	FORMULA	CYCLE MODEL
Dot Product	0x00	A[n], B[n]	Scalar	sum(A[i] * B[i])	vector_len × 2
Elementwise Add	0x01	A[n], B[n]	Vector[n]	A[i] + B[i]	vector_len × 1
ReLU	0x02	A[n]	Vector[n]	max(0, A[i])	vector_len × 1

DATA FLOW PATTERN

Software: Accelerator:

1. Write CONFIG (len, mode) Registers configured
2. Write INPUT_A x N times FIFO A fills
3. Write INPUT_B x N times FIFO B fills
4. Write CONTROL.START = 1 Computation triggers
5. Read STATUS until DONE (synchronous in model)
6. Read OUTPUT x M times FIFO drains results

VALIDATION RESULTS

TEST	OPERATION	INPUT	EXPECTED	STATUS
1	Dot Product	A=[1,2,3,4] B=[5,6,7,8]	70	√ PASS
2	Dot Product (signed)	A=[-1,2,-3,4] B=[5,-6,7,-8]	-70	√ PASS
3	Elementwise Add	A=[10,20,30] B=[1,2,3]	[11, 22, 33]	√ PASS
4	ReLU	A=[5,-3,0,7,-10]	[5, 0, 0, 7, 0]	√ PASS

RESULT

All four tests pass. The signed dot-product case (`-70`, stored as `0xFFFFFFBA`) confirms correct two's-complement handling throughout the stack — essential for quantized neural networks. The C test program written against this VP will run on actual hardware without modification. **Demo 2 passed.**

Demo 3: Recognizing Handwritten Digits

In Demo 2, the custom accelerator proved it does math correctly. This demo runs a real AI program — recognizing handwritten digits (0–9) — with the accelerator doing all the heavy math on the available Virtual Prototype.

HOW THE AI WORKS

Picture of a digit (784 tiny dots — the 28×28 image flattened)

|

Step 1 ► boil the 784 dots down to 64 "clues" (uses ReLU)

|

Step 2 ► turn 64 clues into 10 scores, one per digit (0–9)

|

Pick the highest score ► "this is a 7!"

Every step above is really a big pile of dot products — exactly what the accelerator does.

SHRINKING THE AI

To fit on the small IP, the trained AI is shrunk — a step called quantization — trading a little accuracy for a much smaller, faster program. Full-size accuracy was 97.4%; the shrunk version scored 95.1%, and packed down to just 312 KB.

WHAT HAPPENED

- **All 10 digits, 100% accuracy** — every dot product computed by the accelerator.
- **740 accelerator calls** — over one million simulated cycles, all on one laptop.

HOW IT WAS BUILT

- **Trained on a laptop** — using PyTorch, teaching the AI to recognize digits by showing it many examples until it learned.
- **Prepared for the chip** — compiled the program for the RISC-V processor and loaded it onto the virtual chip.
- **Booted and ran it** — the virtual chip ran the digit recognizer, calling the accelerator for every dot product.

RESULT

Demo 3 passed. **100% accuracy** — a real AI program ran correctly on a chip that doesn't physically exist, using a custom accelerator that also doesn't physically exist. The whole pipeline — train the AI, shrink it, run it on the virtual chip — works end to end.

Crucible: Making VP Workflows Reproducible

VP's power is only realized if VP environments are reproducible. Setting up a VP workflow involves installing specific versions of simulators, cross-compilers, PDKs, and build tools — each with their own dependencies and version constraints. Without infrastructure to manage this, VP becomes "it works on my machine" — powerful for the person who set it up, useless for their colleagues.

Crucible captures the entire VP environment as a reproducible workspace:

VP WORKSPACE

```
REPRODUCE THE PACKAGED WINDOWS DEMO

$ cd virtual-prototyping-mnist-classifier
$ igny workspace sync
$ igny run renode --console \
    scripts/litex_vexriscv_accel.resc -e s --hide-log

crucible.lock pins Renode 1.16.1 (windows-x86_64).
The simulator boots the included kernel and prepared rootfs.
No cross-compiler or model retraining is needed to run.
```

KEY MESSAGE

The entire VP environment — simulator, toolchain versions — is captured in a reproducible workspace. Your colleague clones it and gets the exact same simulation.

The cross-compiler friction illustrates this concretely: Debian/Ubuntu repositories only ship 64-bit RISC-V compilers, and the 32-bit Linux toolchain is otherwise a manual artifact-install exercise. In a Crucible-managed workspace, `igny compiler install riscv32` resolves the correct toolchain, installs it, and binds it to the environment automatically.

Running the Demos

All demos are self-contained. Pre-compiled binaries are already baked into the rootfs — **no cross-compiler is needed to run the demos .**

No physical hardware, FPGA, or paid tooling is required.

PREREQUISITES

REQUIREMENT	DETAIL
Crucible CLI (`igny`)	Available on PATH
Renode	Provided by the workspace via `igny tool install renode`
Host OS	Linux or Windows (tested on Debian and Windows 11)
Cross-compiler	Only needed if rebuilding binaries from source (`igny compiler install riscv32`)
PyTorch	Only needed if retraining the MNIST model; runs in a host Python venv

QUICK START: DEMO 1 — CUSTOM SOFTWARE ON VP

terminal

```
git clone <repository-url>
cd virtual-prototyping-mnist-classifier
igny workspace sync
igny run renode --console scripts/litex_vexriscv_accel.resc -e s --hide-log
# In the guest UART: log in as root (no password)
./hello_world_ignytion
./ignytion_banner
```

EXECUTION ON CRUCIBLE CLI:

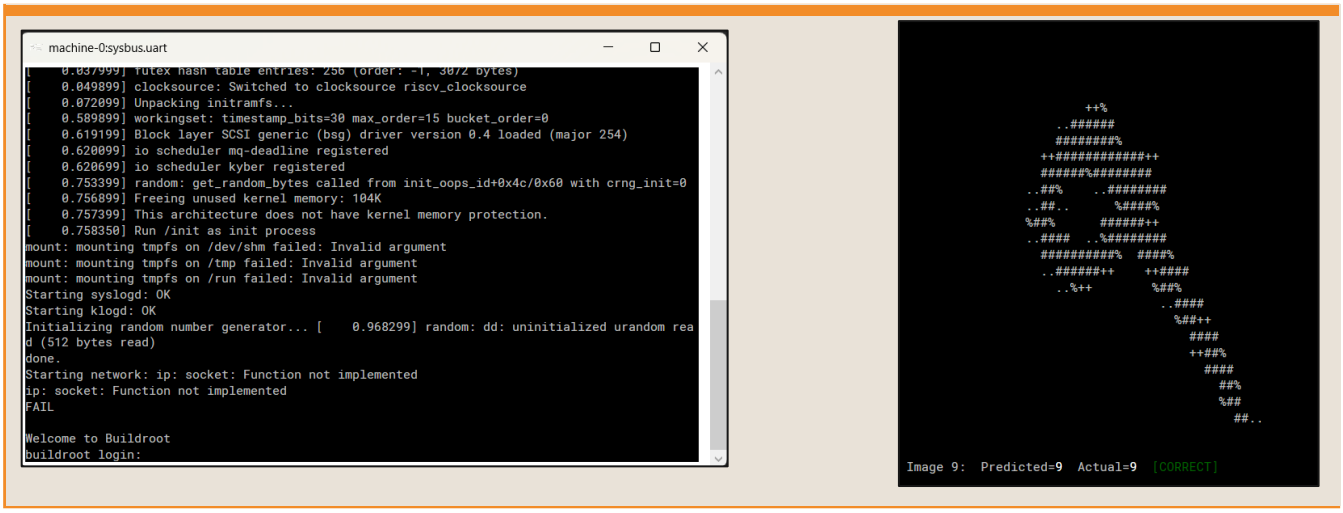
```
$ igny run renode --console scripts/litex_vexriscv_accel.resc -e s --hide-log
```

QUICK START: DEMOS 2 & 3 — ACCELERATOR IP + MNIST CLASSIFIER

terminal

```
# After boot and login as root: root@buildroot:~# ./test_accel
 # accelerator functional tests root@buildroot:~# ./mnist_classify
# MNIST with ASCII art digit display root@buildroot:~# ./mnist_classify -q  # compact mode
```

EXECUTION ON CRUCIBLE CLI:



REBUILDING FROM SOURCE

terminal

```
igny compiler install riscv32
igny run riscv32-buildroot-linux-musl-gcc –static -o test_accel src/test_accel.c
igny run riscv32-buildroot-linux-musl-gcc -static -O2 -o mnist_classify src/mnist_classify.c
```

Conclusion

#	DEMO	CAPABILITY DEMONSTRATED	RESULT
01	Running Custom Software on a Virtual RISC-V Linux System	Full-system OS + cross-compiled software on a virtual SoC	√ PASS — Linux-riscv32
02	Custom AI Accelerator IP Integration	Memory-mapped IP integrated and driven by C MMIO software	√ PASS — 4/4 tests
03	MNIST Neural Network Inference on the Virtual Accelerator	Real quantized neural network running on the virtual IP	√ PASS — 100%, 740 calls

This whitepaper demonstrated a complete, working Virtual Prototyping workflow — from booting Linux on a simulated RISC-V SoC, through integrating a custom AI accelerator IP, to running quantized neural network inference — all without physical hardware. The three progressive demos establish that:

- **VP enables real software development** — Cross-compiled C programs execute on a virtual RISC-V Linux system with full syscall support, ANSI terminal rendering, and standard POSIX interfaces.
- **Custom IP can be prototyped and validated in VP** — A memory-mapped accelerator with standard hardware register conventions was integrated into the SoC and validated with both shell-level and C-level test software.
- **Real workloads run on virtual accelerators** — A quantized MNIST classifier achieved 100% accuracy across 740 accelerator invocations, performing over one million simulated compute cycles.

The path from virtual prototype to physical silicon is continuous. The same VexRiscv core that powered the VP can be taken through open-source RTL-to-GDS flows (Yosys → OpenROAD → Magic → KLayout) on the Skywater 130nm PDK — all orchestrated by Crucible.

Virtual Prototyping is not a novelty. It is the natural way engineering workflows should operate when proper platform infrastructure exists. Crucible makes these workflows accessible, reproducible, and manageable — so that every chip team, from startups to established semiconductor companies, can develop hardware and software in parallel from day one.

Ignytion IO — Build Silicon, Like Software · ignytion.io

© 2026 Igny CLI · Ignytion IO

REFERENCE

Glossary

Key terms used in this whitepaper, ordered by first appearance.

SoC — System-on-a-Chip: an entire small computer (processor, memory, connections) built onto one chip.

RTL — Register-Transfer Level: the detailed hardware design code engineers write before a chip is manufactured.

Tape-out — The moment a chip design is finalized and sent off to be physically manufactured.

VP — Virtual Prototype: a software simulation of a chip that behaves like the real thing, without physical hardware.

RISC-V — A free, open blueprint for a processor — anyone can build a chip using it without paying licensing fees.

Renode — The free simulator program used to run the virtual chip on a laptop.

RTOS — Real-Time Operating System: a lightweight OS for devices that must respond instantly, unlike a full OS like Linux.

Bare-metal firmware — The most basic software that runs directly on a chip, with no operating system underneath it.

MMU — Memory Management Unit: lets a chip run a full OS (like Linux) by organizing memory safely.

Bus (e.g., Wishbone) — The internal "roadway" inside a chip that lets different parts send data to each other.

BSP — Board Support Package: the software layer that lets an OS talk to specific hardware.

Accelerator — A specialized chip part built to do one kind of math (like AI calculations) very fast.

IP (in chip design) — Intellectual Property: a reusable, pre-designed building block dropped into a larger chip design.

Cross-compile — Preparing a program on your laptop so it can run on a different kind of chip (here, RISC-V).

MMIO — Memory-Mapped I/O: software controls a chip part by reading and writing specific memory addresses.

Register — A small "control slot" on a chip part — software writes some to instruct, reads others for answers.

FIFO — First-In-First-Out: a data queue where the first number put in is the first one that comes back out.

Two's complement — The standard method computer hardware uses to represent negative numbers in memory.

Quantization — Shrinking a trained AI model to fit a small chip, trading a little accuracy for smaller size and speed.

MNIST — A famous, beginner-friendly dataset of handwritten digit images (0–9), used to test AI systems.

PyTorch — A popular, free software tool used to build and train AI models.

Argmax — "Pick the biggest score" — the AI's final step of choosing its best guess among possible answers.

Initramfs / rootfs — The small file system loaded into memory that a simulated computer boots from.

Syscall — System call: a request a program makes to the OS, e.g. asking what processor it's running on.

PDK — Process Design Kit: the technical rulebook a manufacturer provides so a design can be built in their factory.

RTL-to-GDS — The full manufacturing pipeline that turns a hardware design into files sent to a chip factory.

Igny / Crucible — Ignytion's toolkit and CLI tool that auto-sets-up the simulator and compiler for consistent demos.

© 2026 Igny CLI · Ignytion IO